GOOGLE AI STUDIO MASTER BUILD DOCUMENT (WITH DIRECT COURIER API)

PROJECT: FASHION AURA ORDER MANAGEMENT SYSTEM (ফ্যাশন আউরা অর্ডার ম্যানেজমেন্ট)

SYSTEM INSTRUCTION FOR GOOGLE AI STUDIO

Build a complete, production-ready, offline-first React + TypeScript + Tailwind CSS + IndexedDB application for "Fashion Aura" (ফ্যাশন আউরা) in Bangladesh, extending it with direct client-side Courier API integration (Steadfast & Pathao) without any intermediate backend server.

STAGE 1: TYPES & DATA MODELS (`src/types/models.ts`)

```
export type OrderType = 'Regular' | 'Pre-order' | 'Free' | 'Exchange';

export type OrderPriority = 'Normal' | 'Urgent';

export type PaymentStatus =

  | 'Unpaid'

  | 'Partially Paid'

  | 'Paid'

  | 'Refund Pending'

  | 'Partially Refunded'

  | 'Refunded';

export type FulfillmentStatus =

  | 'New'

  | 'Confirmed'

  | 'Preparing'
```

```
  | 'Ready to Ship'
  | 'Shipped'
  | 'In Transit'
  | 'Delivered'
  | 'Delivery Failed'
  | 'Return Requested'
  | 'Returned'
  | 'Exchange Processing'
  | 'Cancelled';

export type DeliveryScheduleState = 'Unscheduled' | 'Scheduled';
export type PaymentEntryType = 'advance' | 'payment' | 'refund' | 'adjustment';
export type PaymentMethod = 'Cash/COD' | 'bKash' | 'Nagad' | 'Rocket' | 'Bank' | 'Other';
export type OrderSource = 'Facebook' | 'Meta Business Suite' | 'WhatsApp' | 'Repeat Customer' |
'Other';
export type CourierName = 'Steadfast' | 'Pathao' | 'RedX' | 'Paperfly' | 'Sundarban' | 'Other';
export type ProductSize = '36' | '38' | '40' | '42' | '44' | '46' | '48' | 'Other';
export type SyncState = 'pending' | 'syncing' | 'synced' | 'error';
export type CourierSyncStatus = 'not_sent' | 'queued_offline' | 'sending' | 'sent' | 'failed';

export interface CourierApiConfigRecord {
  courier: CourierName;
  enabled: boolean;
  baseUrl: string;
  apiKey: string;
```

```
  secretKeyOrToken: string;
  storeId?: string;
  updatedAt: string;
}

export interface OrderItemRecord {
  id: string;
  orderId: string;
  productName: string;
  sku?: string;
  size: ProductSize;
  customSizeText?: string;
  quantity: number;
  unitPrice: number;
  color?: string;
  productNotes?: string;
}

export interface OrderRecord {
  id: string;
  orderNumber: string;
  customerId: string;
  customerName: string;
  primaryPhone: string;
```

alternatePhone?: string;

fullAddress: string;

areaVillageStreet?: string;

district: string;

thanaUpazila: string;

landmark?: string;

orderType: OrderType;

priority: OrderPriority;

paymentStatus: PaymentStatus;

fulfillmentStatus: FulfillmentStatus;

orderDate: string;

requestedDeliveryDate?: string;

earliestDispatchDate?: string;

source: OrderSource;

courier: CourierName;

trackingNumber?: string;

courierSyncStatus: CourierSyncStatus;

courierConsignmentId?: string;

courierLastError?: string;

courierSentAt?: string;

subtotal: number;

deliveryCharge: number;

additionalCharge: number;

discount: number;

```
  totalBill: number;
  netPaid: number;
  outstanding: number;
  customerDeliveryInstructions?: string;
  internalStaffNotes?: string;
  isDraft: boolean;
  isDeleted: boolean;
  version: number;
  createdAt: string;
  createdBy: string;
  updatedAt: string;
  updatedBy: string;
  syncState: SyncState;
}

export interface CustomerRecord {
  id: string;
  name: string;
  primaryPhone: string;
  defaultAddress: string;
  district: string;
  thanaUpazila: string;
  orderCount: number;
  totalSpent: number;
```

```
  isRisky: boolean;
}

export interface AuditLogRecord {
  id: string;
  entityType: string;
  entityId: string;
  action: string;
  details: string;
  performedBy: string;
  performedAt: string;
}

export function getDeliveryScheduleState(
  order: Pick<OrderRecord, 'requestedDeliveryDate' | 'earliestDispatchDate'>
): DeliveryScheduleState {
  return Boolean(order.requestedDeliveryDate?.trim() || order.earliestDispatchDate?.trim())
    ? 'Scheduled'
    : 'Unscheduled';
}
```

STAGE 2: DATABASE SETUP (`src/db/database.ts`)

```
import Dexie, { type Table } from 'dexie';
import type {
  OrderRecord,
  OrderItemRecord,
  CustomerRecord,
  AuditLogRecord,
  CourierApiConfigRecord
} from '../types/models';

export class FashionAuraDB extends Dexie {
  orders!: Table<OrderRecord, string>;
  orderItems!: Table<OrderItemRecord, string>;
  customers!: Table<CustomerRecord, string>;
  auditLogs!: Table<AuditLogRecord, string>;
  courierConfigs!: Table<CourierApiConfigRecord, string>;

  constructor() {
    super('FashionAuraCourierDB');
    this.version(2).stores({
      orders:
        'id, &orderNumber, customerId, primaryPhone, orderType, priority, paymentStatus, fulfillmentStatus, orderDate, requestedDeliveryDate, earliestDispatchDate, district, courier, courierSyncStatus, isDraft, isDeleted, syncState, updatedAt',
      orderItems: 'id, orderId, productName, sku, size',
```

```
      customers: 'id, &primaryPhone, name, district, isRisky',
      auditLogs: 'id, entityType, entityId, performedAt',
      courierConfigs: 'courier, enabled'
    });
  }
}

export const db = new FashionAuraDB();

export async function ensureDefaultCourierConfigs(): Promise<void> {
  const count = await db.courierConfigs.count();
  if (count > 0) return;
  const now = new Date().toISOString();
  await db.courierConfigs.bulkPut([
    {
      courier: 'Steadfast',
      enabled: false,
      baseUrl: 'https://portal.packzy.com/api/v1',
      apiKey: '',
      secretKeyOrToken: '',
      updatedAt: now
    },
    {
      courier: 'Pathao',
```

```
      enabled: false,
      baseUrl: 'https://api-hermes.pathao.com',
      apiKey: '',
      secretKeyOrToken: '',
      storeId: '',
      updatedAt: now
    }
  ]);
}
```

STAGE 3: DIRECT CLIENT-SIDE COURIER API SERVICE (`src/services/courierApiService.ts`)

```
import { db } from '../db/database';
import type { OrderRecord, OrderItemRecord } from '../types/models';

export interface CourierPushResult {
  orderId: string;
  orderNumber: string;
  status: 'sent' | 'queued_offline' | 'failed';
  trackingCode?: string;
  consignmentId?: string;
  message: string;
```

```
}

export class CourierAPIService {
  static async sendOrderToCourier(
    orderId: string,
    options?: { allowEarlyDispatchOverride?: boolean }
  ): Promise<CourierPushResult> {
    const order = await db.orders.get(orderId);
    if (!order) throw new Error('Order not found');

    if (order.courierSyncStatus === 'sent' && order.trackingNumber) {
      return {
        orderId: order.id,
        orderNumber: order.orderNumber,
        status: 'sent',
        trackingCode: order.trackingNumber,
        consignmentId: order.courierConsignmentId,
        message: `Already sent (Tracking: ${order.trackingNumber})`
      };
    }

    const todayIso = new Date().toISOString().slice(0, 10);
    if (
      order.earliestDispatchDate &&
```

```
  todayIso < order.earliestDispatchDate &&
  !options?.allowEarlyDispatchOverride
) {
  return {
    orderId: order.id,
    orderNumber: order.orderNumber,
    status: 'failed',
    message: `Restricted: Do Not Dispatch Before ${order.earliestDispatchDate}`
  };
}

if (!navigator.onLine) {
  await db.orders.update(order.id, {
    courierSyncStatus: 'queued_offline',
    courierLastError: 'Queued offline — waiting for internet',
    updatedAt: new Date().toISOString()
  });
  return {
    orderId: order.id,
    orderNumber: order.orderNumber,
    status: 'queued_offline',
    message: 'Phone is offline. Added to Pending Courier Sync queue.'
  };
}
```

```
    const config = await db.courierConfigs.get(order.courier);
    if (!config || !config.enabled) {
      const msg = `${order.courier} API is not enabled in Settings.`;
      await db.orders.update(order.id, {
        courierSyncStatus: 'failed',
        courierLastError: msg,
        updatedAt: new Date().toISOString()
      });
      return { orderId: order.id, orderNumber: order.orderNumber, status: 'failed', message: msg };
    }

    await db.orders.update(order.id, {
      courierSyncStatus: 'sending',
      courierLastError: undefined,
      updatedAt: new Date().toISOString()
    });

    const items = await db.orderItems.where('orderId').equals(order.id).toArray();

    try {
      let result: { trackingCode: string; consignmentId: string };
      if (order.courier === 'Steadfast') {
        result = await this.pushToSteadfast(order, items, config.baseUrl, config.apiKey,
config.secretKeyOrToken);
```

```
    } else if (order.courier === 'Pathao') {
      result = await this.pushToPathao(order, items, config.baseUrl, config.secretKeyOrToken,
config.storeId || '');
    } else {
      throw new Error(`Direct API for ${order.courier} not supported.`);
    }

    const now = new Date().toISOString();
    await db.transaction('rw', [db.orders, db.auditLogs], async () => {
      await db.orders.update(order.id, {
        courierSyncStatus: 'sent',
        trackingNumber: result.trackingCode,
        courierConsignmentId: result.consignmentId,
        courierSentAt: now,
        courierLastError: undefined,
        fulfillmentStatus:
          order.fulfillmentStatus === 'New' || order.fulfillmentStatus === 'Confirmed'
            ? 'Ready to Ship'
            : order.fulfillmentStatus,
        updatedAt: now
      });
      await db.auditLogs.add({
        id: crypto.randomUUID(),
        entityType: 'courier_api',
        entityId: order.id,
```

```
        action: `SENT_${order.courier.toUpperCase()}`,
        details: `Tracking: ${result.trackingCode}`,
        performedBy: 'Admin',
        performedAt: now
      });
    });

    return {
      orderId: order.id,
      orderNumber: order.orderNumber,
      status: 'sent',
      trackingCode: result.trackingCode,
      consignmentId: result.consignmentId,
      message: `Sent to ${order.courier}! Tracking: ${result.trackingCode}`
    };
  } catch (err) {
    const errorMsg = err instanceof Error ? err.message : 'Courier API Error';
    await db.orders.update(order.id, {
      courierSyncStatus: 'failed',
      courierLastError: errorMsg,
      updatedAt: new Date().toISOString()
    });
    return { orderId: order.id, orderNumber: order.orderNumber, status: 'failed', message: errorMsg };
  }
```

```
  }

  static async bulkSyncOrders(orderIds?: string[]): Promise<{ sentCount: number; failedCount: number; queuedCount: number }> {
    let targetOrders: OrderRecord[];
    if (orderIds && orderIds.length > 0) {
      const fetched = await db.orders.bulkGet(orderIds);
      targetOrders = fetched.filter((o): o is OrderRecord => Boolean(o && !o.isDeleted));
    } else {
      targetOrders = await db.orders
        .filter((o) => !o.isDeleted && (o.courierSyncStatus === 'queued_offline' || o.courierSyncStatus === 'failed'))
        .toArray();
    }

    let sentCount = 0, failedCount = 0, queuedCount = 0;
    for (const order of targetOrders) {
      const res = await this.sendOrderToCourier(order.id);
      if (res.status === 'sent') sentCount++;
      else if (res.status === 'queued_offline') queuedCount++;
      else failedCount++;
    }
    return { sentCount, failedCount, queuedCount };
  }
```

```
  private static async pushToSteadfast(
    order: OrderRecord,
    items: OrderItemRecord[],
    baseUrl: string,
    apiKey: string,
    secretKey: string
  ): Promise<{ trackingCode: string; consignmentId: string }> {
    if (!apiKey.trim() || !secretKey.trim()) throw new Error('Steadfast API Key and Secret Key required.');
    const itemSummary = items.map((i) => `${i.productName} (Size ${i.size} x${i.quantity})`).join(', ');

    const res = await fetch(`${baseUrl.replace(/\/$/, '')}/create_order`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Api-Key': apiKey.trim(),
        'Secret-Key': secretKey.trim()
      },
      body: JSON.stringify({
        invoice: order.orderNumber,
        recipient_name: order.customerName,
        recipient_phone: order.primaryPhone,
        recipient_address: `${order.fullAddress}, ${order.thanaUpazila}, ${order.district}`,
        cod_amount: Math.max(0, Math.round(order.outstanding)),
```

```
      note: itemSummary
    })
  });
  const data = await res.json().catch(() => null);
  if (!res.ok || (data && data.status && data.status !== 200)) {
    throw new Error(data?.message || `HTTP ${res.status}`);
  }
  const trackingCode = String(data?.consignment?.tracking_code ||
data?.consignment?.consignment_id || '').trim();
  if (!trackingCode) throw new Error('Steadfast response missing tracking_code');
  return { trackingCode, consignmentId: String(data?.consignment?.consignment_id ||
trackingCode) };
 }


 private static async pushToPathao(
  order: OrderRecord,
  items: OrderItemRecord[],
  baseUrl: string,
  bearerToken: string,
  storeId: string
 ): Promise<{ trackingCode: string; consignmentId: string }> {
  if (!bearerToken.trim() || !storeId.trim()) throw new Error('Pathao Token and Store ID
required.');
  const totalQty = items.reduce((sum, i) => sum + Math.max(1, i.quantity), 0);
  const itemDesc = items.map((i) => `${i.productName} (Size ${i.size})`).join(', ');
```

```
const res = await fetch(`${baseUrl.replace(/\/$/, '')}/aladdin/api/v1/orders`, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    Accept: 'application/json',
    Authorization: `Bearer ${bearerToken.trim()}`
  },
  body: JSON.stringify({
    store_id: Number(storeId),
    merchant_order_id: order.orderNumber,
    recipient_name: order.customerName,
    recipient_phone: order.primaryPhone,
    recipient_address: `${order.fullAddress}, ${order.thanaUpazila}, ${order.district}`,
    delivery_type: 48,
    item_type: 2,
    item_quantity: totalQty || 1,
    item_weight: 0.5,
    amount_to_collect: Math.max(0, Math.round(order.outstanding)),
    item_description: itemDesc
  })
});
const data = await res.json().catch(() => null);
if (!res.ok) throw new Error(data?.message || `HTTP ${res.status}`);
const consignmentId = String(data?.data?.consignment_id || '').trim();
```

```
    if (!consignmentId) throw new Error('Pathao response missing consignment_id');
    return { trackingCode: consignmentId, consignmentId };
  }
}
```

STAGE 4: UTILS, FORMATTERS & TEMPLATE ENGINE (`src/utils/coreUtils.ts` & `src/services/smartPasteAndCourier.ts`)

```
const EN_TO_BN: Record<string, string> = { '0': '০', '1': '১', '2': '২', '3': '৩', '4': '৪', '5': '৫', '6': '৬', '7': '৭', '8': '৮', '9': '৯' };
const BN_TO_EN: Record<string, string> = { '০': '0', '১': '1', '২': '2', '৩': '3', '৪': '4', '৫': '5', '৬': '6', '৭': '7', '৮': '8', '৯': '9' };

export function toEnglishDigits(s: string): string { return (s || '').replace(/[০-৯]/g, (c) => BN_TO_EN[c] ?? c); }
export function formatDigits(val: string | number, useBn: boolean): string {
  const s = String(val ?? '');
  if (!useBn) return toEnglishDigits(s);
  return s.replace(/[0-9]/g, (c) => EN_TO_BN[c] ?? c);
}
export function formatBDT(amt: number, useBn: boolean): string {
  const r = Math.round(amt || 0);
  return `${r < 0 ? '-' : ''}৳${formatDigits(Math.abs(r).toLocaleString('en-IN'), useBn)}`;
}
```

```
export function normalizeBDPhone(raw: string): string {
  const a = toEnglishDigits(raw).replace(/[\s\-().]/g, '');
  if (a.startsWith('+880')) return '0' + a.slice(4);
  if (a.startsWith('880') && a.length === 13) return '0' + a.slice(3);
  return a;
}
export function isValidBDPhone(raw: string): boolean { return
/^01[3-9]\d{8}$/.test(normalizeBDPhone(raw)); }
export function parseIntSafe(raw: string): number {
  const c = toEnglishDigits(raw).replace(/[^0-9]/g, '');
  return c ? parseInt(c, 10) : 0;
}
export function generateOrderId(): string {
  const now = new Date();
  return `FA-${String(now.getFullYear()).slice(-2)}${String(now.getMonth() + 1).padStart(2,
'0')}${String(now.getDate()).padStart(2, '0')}-${Math.floor(1000 + Math.random() * 9000)}`;
}


export function calculateFinancials(params: { orderType: string; items: Array<{ unitPrice:
number; quantity: number }>; deliveryCharge: number; additionalCharge: number; discount:
number; netPaid: number; }) {
  const subtotal = params.items.reduce((s, i) => s + Math.max(0, Math.round(i.unitPrice)) *
Math.max(1, Math.round(i.quantity)), 0);
  const totalBill = params.orderType === 'Free' ? 0 : Math.max(0, subtotal +
Math.round(params.deliveryCharge) + Math.round(params.additionalCharge) -
Math.round(params.discount));
  const outstanding = totalBill - Math.round(params.netPaid);
```

```
  return { subtotal, totalBill, outstanding: Math.max(0, outstanding), paymentStatus:
params.orderType === 'Free' || (totalBill > 0 && params.netPaid >= totalBill) ? 'Paid' :
params.netPaid > 0 ? 'Partially Paid' : 'Unpaid' };

}


export const BD_DISTRICTS = ['Dhaka', 'Chattogram', 'Gazipur', 'Narayanganj', 'Sylhet',
'Rajshahi', 'Khulna', 'Barishal', 'Rangpur', 'Mymensingh', 'Cumilla', 'Bogura'];


export function parseOrderMessageOffline(rawText: string) {

  const digits = toEnglishDigits(rawText);

  const phone = (digits.match(/(?:\+?880|0)1[3-9]\d[\d\s\-]{7,10}/) || [])[0];

  const price = (digits.match(/(?:price|দাম|মূল্য|tk|৳)\s*[:\-=]?\s*(\d{3,5})/i) || [])[1];

  return {

    customerName: { value: rawText.split('\n')[0] || '', status: 'Detected' },

    primaryPhone: { value: phone ? normalizeBDPhone(phone) : '', status: phone ? 'Detected' :
'Not found' },

    fullAddress: { value: rawText, status: 'Detected' },

    district: { value: 'Dhaka', status: 'Detected' },

    productName: { value: 'Three Piece', status: 'Detected' },

    size: { value: '40' as const, status: 'Detected' },

    unitPrice: { value: price ? parseInt(price, 10) : 1250, status: price ? 'Detected' : 'Not found' },

    deliveryCharge: { value: 80, status: 'Detected' },

    advancePaid: { value: 0, status: 'Not found' },

    deliveryInstructions: { value: '', status: 'Not found' }

  };

}
```

```
export function generateCourierText(order: OrderRecord, items: OrderItemRecord[], lang: 'bn' | 'en' = 'bn'): string {
  const itemsText = items.map((i) => `${i.productName} (Size: ${i.size} × ${i.quantity})`).join(', ');
  const cod = Math.max(0, order.outstanding);
  if (lang === 'bn') {
    return [
      `অর্ডার আইডি: ${order.orderNumber}`,
      `নাম: ${order.customerName}`,
      `মোবাইল: ${order.primaryPhone}`,
      `ঠিকানা: ${order.fullAddress}, ${order.thanaUpazila}, ${order.district}`,
      `পণ্য: ${itemsText}`,
      `মোট বিল: ৳${order.totalBill} (ডেলিভারি: ৳${order.deliveryCharge})`,
      `COD: ${cod === 0 ? 'প্রিপেইড (৳০)' : `৳${cod}`}`
    ].join('\n');
  }
  return [
    `Order ID: ${order.orderNumber}`,
    `Customer: ${order.customerName}`,
    `Phone: ${order.primaryPhone}`,
    `Address: ${order.fullAddress}, ${order.district}`,
    `Items: ${itemsText}`,
    `Total: BDT ${order.totalBill} | COD: BDT ${cod}`
  ].join('\n');
}
```

STAGE 5: SETTINGS PANEL & APP ENTRY (`src/components/CourierSettingsPanel.tsx` & `src/App.tsx`)

CourierSettingsPanel:

```
import React, { useEffect, useState } from 'react';
import { ShieldCheck, Save, Truck } from 'lucide-react';
import { db, ensureDefaultCourierConfigs } from '../db/database';
import type { CourierApiConfigRecord } from '../types/models';

export const CourierSettingsPanel: React.FC<{ lang: 'bn' | 'en'; onSaved: (m: string) => void }> = ({ lang, onSaved }) => {
  const [configs, setConfigs] = useState<CourierApiConfigRecord[]>([]);
  useEffect(() => { ensureDefaultCourierConfigs().then(async () => setConfigs(await db.courierConfigs.toArray())); }, []);

  const handleChange = (courier: string, field: string, val: any) => {
    setConfigs(prev => prev.map(c => c.courier === courier ? { ...c, [field]: val } : c));
  };

  const handleSave = async (cfg: CourierApiConfigRecord) => {
    await db.courierConfigs.put({ ...cfg, updatedAt: new Date().toISOString() });
    onSaved(`${cfg.courier} API settings saved locally!`);
  };
```

```
  return (
    <div className="space-y-4">
      <div className="p-3 bg-amber-50 border border-amber-300 text-xs rounded-xl">
        <ShieldCheck className="w-4 h-4 inline mr-1 text-amber-800" />
        {lang === 'bn' ? 'এপিআই কি আপনার ফোনের IndexedDB-তে সুরক্ষিত থাকে।' : 'API keys stored securely in local IndexedDB.'}
      </div>
      {configs.map(cfg => (
        <div key={cfg.courier} className="bg-white p-4 rounded-2xl border space-y-3 shadow-sm">
          <div className="flex justify-between items-center border-b pb-2">
            <span className="font-bold text-pink-900 flex items-center gap-1.5"><Truck className="w-4 h-4" />{cfg.courier} API</span>
            <label className="text-xs font-bold flex items-center gap-1 cursor-pointer">
              <input type="checkbox" checked={cfg.enabled} onChange={e => handleChange(cfg.courier, 'enabled', e.target.checked)} />
              {cfg.enabled ? 'Enabled' : 'Disabled'}
            </label>
          </div>
          <div className="space-y-2 text-xs">
            <input type="url" value={cfg.baseUrl} onChange={e => handleChange(cfg.courier, 'baseUrl', e.target.value)} placeholder="Base URL" className="w-full p-2 border rounded font-mono" />
            {cfg.courier === 'Pathao' && (
              <input type="text" value={cfg.storeId || ''} onChange={e => handleChange(cfg.courier, 'storeId', e.target.value)} placeholder="Store ID" className="w-full p-2 border rounded font-mono" />
```

```
          )}

          <input type="password" value={cfg.apiKey || cfg.secretKeyOrToken} onChange={e =>
handleChange(cfg.courier, cfg.courier === 'Steadfast' ? 'apiKey' : 'secretKeyOrToken',
e.target.value)} placeholder="API Key / Token" className="w-full p-2 border rounded
font-mono"/>

        </div>

        <button onClick={() => handleSave(cfg)} className="px-3 py-2 bg-pink-800 text-white
font-bold text-xs rounded-xl flex items-center gap-1"><Save className="w-3.5 h-3.5"/>Save
{cfg.courier}</button>

      </div>

    ))}

  </div>

 );

};
```